SOFTWARE ENGINEERING · LAB WALKTHROUGH

Container to Cluster

Docker, a local AWS emulator, and Kubernetes — with no cloud account and nothing to spend.

Nivah · [course / section] · [date]

THE ASSIGNMENT

A starter app, a container, a cluster

Students get a small Task Tracker API already written. Their work is packaging it, deploying it, and keeping it running — so the time goes to Docker and Kubernetes, not to writing endpoints.

3
phases, each building on the last

4–6
hours of work, due in 48

$0
no AWS account, no cloud spend

WHY AN EMULATOR

A cluster per student, for nothing

Real EKS in a course

- About $0.10 per hour, per cluster
- An AWS account and billing for each student
- IAM permissions to hand out and revoke
- Forgotten resources that keep charging

Floci on a laptop

- Open source, no sign-up, no auth token
- Serves the AWS APIs on localhost:4566
- EKS starts a real k3s node in Docker
- Delete it and nothing lingers

HOW IT FITS TOGETHER

One laptop, four moving parts

aws cli
Calls the EKS API that Floci serves on port 4566

→

floci
Starts a real k3s node in a Docker container

→

kubectl
Reaches that cluster on port 6500

→

2 pods
The student image, running and serving

The image never goes through a registry. It is exported from Docker and loaded straight into the node, which sidesteps the networking between the emulator and the cluster.

PHASE 1 · ABOUT 1.5 HOURS

Containerize

What they write

- A Dockerfile on a slim base image
- A non-root user for the process
- A .dockerignore for the build context
- A container that serves on port 8080

```
$ docker build -t task-tracker:v1 .
$ docker run -p 8080:8080 task-tracker:v1
$ curl localhost:8080/health
{"status":"ok"}
```

PHASE 2 · ABOUT 2 HOURS

Deploy to the cluster

What they write

- A Deployment with two replicas
- A readiness probe on /health
- A Service in front of the pods
- Config passed in as an environment variable

```
$ aws eks create-cluster ...
$ kubectl get nodes
Ready control-plane v1.34.1
$ kubectl apply -f k8s/
```

PHASE 3 · ABOUT 1 HOUR

Operate it

Roll forward

Ship v2 and watch the replicas swap over without dropping traffic.

Roll back

Undo the release and confirm the old version is serving again.

Break it

Delete a pod and watch a replacement appear in seconds.

Three commands, but the first time a deleted pod comes back on its own is usually the moment Kubernetes stops feeling like a deployment script.

From nothing to a running cluster

```
brew install floci-io/floci/floci

floci start && eval $(floci env)

aws eks create-cluster --name lab-cluster \
  --role-arn arn:aws:iam::000000000000:role/eks-role \
  --resources-vpc-config subnetIds=subnet-default-us-east-1-a

docker exec floci-eks-lab-cluster \
  cat /etc/rancher/k3s/k3s.yaml | sed s/6443/6500/ \
  > ~/.kube/floci-lab.yaml

export KUBECONFIG=~/.kube/floci-lab.yaml

kubectl get nodes

Ready   control-plane   v1.34.1
```

Ship it, then break it

DEPLOY

```
docker build -t task-tracker:v1 .

docker save task-tracker:v1 \
  | docker exec -i floci-eks-lab-cluster ctr \
  -n k8s.io images import -

kubectl apply -f k8s/

kubectl port-forward svc/task-tracker 8080:80
```

OPERATE

```
kubectl rollout status deploy/task-tracker

kubectl rollout undo deploy/task-tracker

kubectl delete pod task-tracker-abc123

kubectl get pods

a replacement, seconds old
```

The v2 image repeats the deploy column with a new tag. Every image has to be loaded into the node again — nothing is pulled.

WHAT THEY HAND IN

Files and evidence, per phase

Phase	Files they write	Evidence
Containerize	Dockerfile and .dockerignore	Screenshot of health, version and a task POST
Deploy	deployment.yaml and service.yaml, filled in from skeletons	Pods and service listed, plus a working request
Operate	No new files — the same manifests, edited	Rollout, rollback, and a pod list before and after

THE WRITTEN PART

Half a page, three questions

1. What is the difference between an image, a container, and a pod?
2. Why did deleting a pod not take your app down?
3. Name one thing EKS manages on real AWS, and one thing you would still own.

How it arrives

A repo link with the Dockerfile, the manifests and a README — plus one PDF holding every screenshot and the short answers.

One PDF grades far faster than loose image files.

Stated up front: a student blocked by their setup can submit the exact error, what they tried and what they ruled out, and still earn most of that phase.

TESTING THE LAB MYSELF

Seven failures, none of them Kubernetes

What I saw	What it actually was
the Dockerfile cannot be empty	Files saved with no contents
Subnet does not exist	The documented example subnet was fictional
provide credentials	Token auth the local cluster rejects
503 Service Unavailable	The built-in registry never forwarded
InvalidImageName	A stream edit mangled one line of YAML
No help topic for 'ctr'	A missing subcommand in the node image
0/1 Running	Two typos in the app, caught one at a time

THE ACCIDENTAL LESSON

Two typos, two kinds of broken

LOUD FAILURE

The pod never went ready

A misspelled function made the health endpoint return a 500. The readiness probe refused the pod, the rollout stalled, and the old version kept serving every request.

Nobody was affected. The system stopped itself.

QUIET FAILURE

The pod was healthy and wrong

A space instead of an underscore meant the app read a variable that did not exist. Health checks passed, both replicas ran, and the version endpoint quietly answered "unknown".

Nothing alerted. Only reading the output caught it.

THE TAKEAWAY

Students learn to describe a system's desired state — and to diagnose the gap when reality disagrees.

Everything memorable in the lab comes from one idea. You declare what you want, and a controller keeps closing the distance between that and what exists. The self-healing pod and the stalled rollout are the same loop doing its job.

BEING HONEST ABOUT SCOPE

They learn Kubernetes, not EKS

What transfers anywhere

- Images, manifests and kubectl
- Deployments, services, probes
- Rollouts, rollbacks, self-healing
- The debugging escalation path

What they never touch

- A managed, multi-AZ control plane
- IAM mapped to cluster identities
- Node groups, scaling, VPC networking
- Load balancers and what any of it costs

One reflection question keeps this honest: name something you did here that would be genuinely different on real AWS.

HOW HARD IS IT

The concepts are easy. The environment is not.

4

Concepts

A Dockerfile, two YAML files, seven commands

8

Environment

Setup, ports, contexts, terminal state

6

Overall

Fair for a junior or senior course

The risk is not a student running out of work. It is one student stuck on a single setup error at hour 40 with nobody to ask.

WHAT THE DRY RUN CHANGED

Four edits to the handout

Exact values, not examples

The subnet and cluster commands are now the ones that actually work on this emulator.

One image path, tested

The registry route is dropped. The image is loaded into the node with a single given command.

Full files, no edit tricks

Students get complete manifests to fill in, because a mangled one-line edit cost me an hour.

One command per terminal

A shell alias restores the environment, since every new tab loses it and every student will hit that.

WHERE IT STANDS

Tested end to end, on one machine, by one person who kept getting it wrong.

Next: a starter repo students can clone, a run on Windows and Intel hardware, and a deliberately broken build in phase 3 so the debugging is assigned rather than accidental.

Questions welcome — especially about what would break on your machine